

HTML-CSS

Section 1.3 : HTML/CSS allez pus loin

20 octobre 2024

À la fin de ce cours, l'apprenant doit :

- ☞ Maîtriser les langages HTML et CSS
- ☞ Créer des pages web en HTML et CSS
- ☞ Structurer une page web
- ☞ Transformer une maquette en page web

le cours nécessite d'aucun pré-requis.

Refs : <https://openclassrooms.com/fr/courses/1603881-creez-votre-site-web-avec-html5-et-css3/8061253-tirez-un-maximum-de-ce-cours>

CSS plus Iain

Structuration de votre page

Balise d'en-tête : `<header>`

-  C'est l'en-tête de la page. En général, on trouve les logos, des bannières, ...

Balise de navigation : `<nav>`

-  Cette balise regroupe tous les principaux liens de navigation du site. On y place par exemple le menu principal de votre site

Balise pied de page : `<footer>`

-  C'est une balise pour spécifier le pied de page. En général, on met des informations comme des liens de contact, les mentions légales, la politique de confidentialité, etc.

Structuration votre page

👉 Balise de contenu : `<main>`

☞ Dans cette balise, nous avons le contenu de la page qu'on peut sub-diviser en sections

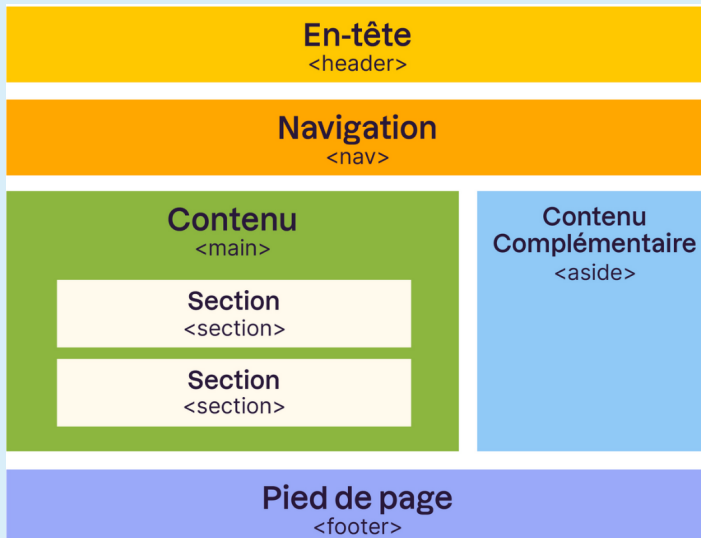
👉 Balise pour les sections : `<section>`

☞ Cette balise sert à regrouper des **contenus** en fonction de leur thématique. Elle englobe généralement une portion du contenu au centre de la page.

👉 Balise de contenu complémentaire : `<aside>`

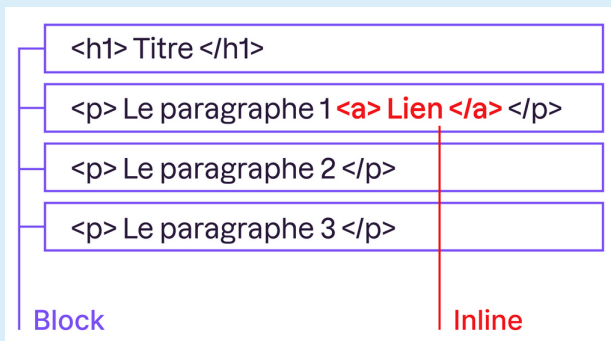
☞ Cette balise contient des informations complémentaires au document que l'on visualise

Structuration votre page



Les balises de type block et inline

- 👉 **block** : elle crée automatiquement un retour à la ligne avant et après ;
- 👉 **inline** : ce type se trouve obligatoirement à l'intérieur d'une balise **block** .

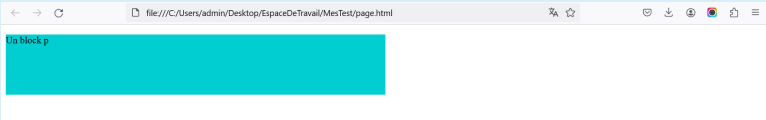


Dimension des balises de type **block**

- ☞ Les balises de type block peuvent être dimensionnées suivant :
 - ☞ sa largeur avec l'attribut **width**
 - ☞ sa hauteur avec l'attribut **height**
- ☞ Les valeurs de ces attributs sont fixées en **px** ou en **%**
- ☞ NB : Le block père va servir de référence.

Dimension des balises de type **block**

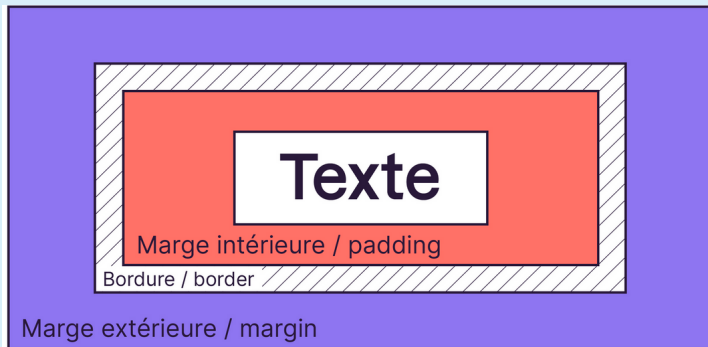
```
p{  
  width: 50%;  
  height: 100px;  
  background-color: darkturquoise;  
}
```



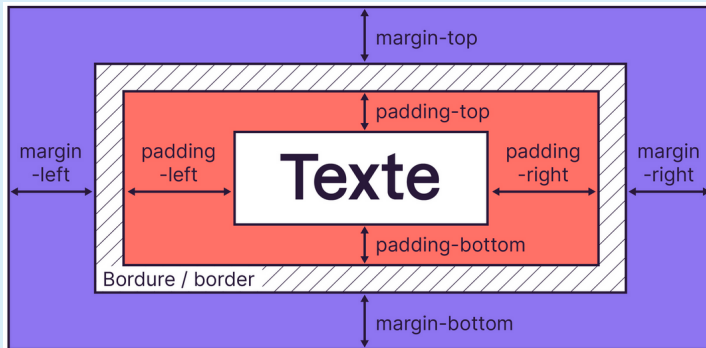
Marges intérieure et extérieure **margin et padding**

- ☞ Les balises de type block peuvent avoir des marges intérieure comme extérieure
- ☞ **margin** et **padding** gère les quatres cotés (haut, gauche, droite et bas)
- ☞ Il y a une possibilité les gérer séparément :
 - ☞ **margin** et **padding** gèrent les quatres cotés (haut, gauche, droite et bas)
 - ☞ margin-top, margin-left, margin-right et margin-bottom
 - ☞ padding-top, padding-left, padding-right et padding-bottom
- ☞ Les valeurs de ces attributs sont fixées en **px** ou en **%**

Marges intérieure et extérieure **margin et padding**



Marges intérieure et extérieure **margin et padding**



- ☞ La mise en page avec flex se fait :
 - ☞ en définissant le conteneur
 - ☞ placer les éléments dans ce conteneur
- ☞ Pour l'alignement, il faut lancer d'abord **display : flex**
- ☞ après choisir l'orientation avec le paramètre **flex-direction** qui peut prendre les valeurs ci-près :
 - ☞ **row** : horizontal (par défaut) ;
 - ☞ **column** : vertical ;
 - ☞ **row-reverse** : horizontal et renverser ;
 - ☞ **column-reverse** : vertical et renverser.

- 👉 le retour à la ligne des éléments se fait par : **flex-wrap**
 - ☞ **nowrap** : sur la même ligne (par défaut)
 - ☞ **wrap** : lorsqu'il y a plus de place, les autres éléments sont à la ligne suivante
 - ☞ **wrap-reverse** : lorsqu'il y a plus de place, les autres éléments sont à la ligne suivante en sens inverse

Axes principal et secondaire

- ☞ Les éléments sont organisés par défaut de manière horizontale mais ils peuvent être organisés de manière vertical avec le paramètre **flex-direction**
 - ☞ si vos éléments sont organisés horizontalement, l'axe secondaire est l'axe vertical ;
 - ☞ si vos éléments sont organisés verticalement, l'axe secondaire est l'axe horizontal.

Axe principal

- ☞ Pour changer leur alignement : **justify-content**
 - ☞ **flex-start** : alignés au début (par défaut) ;
 - ☞ **flex-end** : alignés à la fin ;
 - ☞ **center** : alignés au centre ;
 - ☞ **space-between** : les éléments sont étirés sur tout l'axe (il y a de l'espace entre eux) ;
 - ☞ **space-around** : idem, les éléments sont étirés sur tout l'axe, mais ils laissent aussi de l'espace sur les extrémités.

Axe secondaire

- ✎ Pour changer leur alignement : **align-items**
 - ☞ **stretch** : éléments étirés sur tout l'axe (valeur par défaut) ;
 - ☞ **flex-start** : éléments alignés au début ;
 - ☞ **flex-end** : éléments alignés à la fin ;
 - ☞ **center** : éléments alignés au centre ;
 - ☞ **baseline** : alignés sur la ligne de base (comme **flex-start**)

Exemple

☞ Les éléments du menu sont définis par la balise **ul**

```
<ul class="menu">  
  <li><a href="#">Accueil</a></li>  
  <li><a href="#">Menu 1</a></li>  
  <li><a href="#">Menu 2</a></li>  
  <li><a href="#">Menu 3</a></li>  
  <li><a href="#">Contacts</a></li>  
</ul>
```

Exemple

- ☞ Mettre un cadre, un fond et supprimer les marges intérieures depuis la feuille CSS

```
<> page.html # feuilleCSS.css X
1  .menu {
2      border: 1px solid green;
3      background-color: #ccc ;
4      padding: 0px;
5  }
```

Exemple

- ☞ Supprimer les puces des balises `li` qui sont dans la class `.menu`

```
7  .menu li {  
8      list-style-type: none;  
9  }
```

Exemple

- ➡ Ajouter des bordures aux liens **a**
- ➡ Transformer les liens du menu en éléments de type bloc
- ➡ Ajouter leur également une marge interne.

```
.menu a {  
  border: 1px solid red;  
  display: block;  
  padding: 0.5rem;  
}
```

Exemple

👉 Définir la liste **flexbox**

```
page.html # feuilleCSS.css
1 .menu {
2     border: 1px solid green;
3     background-color: #ccc ;
4     padding: 0;
5     display: flex;
6 }
7
```

Exemple

- ➡ Espacement des éléments du menu : `justify-content : space-around ;`
- ➡ Redimensionner les éléments du menu afin qu'ils aient tous la même taille :
 - ➡ `flex-basis : 120px ; /* Ou avec width : 120px ; */`
 - ➡ `text-align : center ;`

Exemple

👉 Ajuster les apparences

```
@import url("https://fonts.googleapis.com/css?family=Roboto:400");
body {
  font-family: Roboto, sans-serif;
}
.menu {
  background-color: #999;
  padding: 0;
  display: flex;
  justify-content: center;
  border-bottom: 3px solid #333;
}
```

Exemple

👉 Ajuster les apparences

```
.menu li {  
  list-style-type: none;  
  flex-basis: 120px;  
  text-align: center;  
  margin: 0 0.3rem;  
}  
  
.menu a {  
  display: block;  
  padding: 0.5rem;  
  background-color: #aaa;  
  text-decoration: none;  
  color: #111;  
}
```

Exemple

- ✎ Modifier les apparences du lien activé : `a :active`
- ✎ Modifier les apparences du lien où se trouve le cursor :
`a :hover`
- ✎ etc.

Gestion des menus avec **grid**

- ☞ De la même que **flex**, on peut faire la mise en forme avec **display : grid ;**
- ☞ On peut définir des colonnes
 - ☞ **grid-template-columns** : nombre de colonnes et la largeur de chacune.
 - ☞ **grid-template-rows** : nombre de lignes et la hauteur de chacune.

grid-template-columns

☞ Soit la page HTML avec 9 éléments

```
<div class="conteneur">  
  <div class="box">Élément 1</div>  
  <div class="box">Élément 2</div>  
  <div class="box">Élément 3</div>  
  <div class="box">Élément 4</div>  
  <div class="box">Élément 5</div>  
  <div class="box">Élément 6</div>  
  <div class="box">Élément 7</div>  
  <div class="box">Élément 8</div>  
  <div class="box">Élément 9</div>  
</div>
```

grid-template-columns



```
✓ .box {  
  height: 150px;  
}  
✓ .conteneur {  
  display: grid;  
  grid-template-columns: 200px 200px 200px;  
}
```

grid-template-rows

- ✎ Pour supprimer la hauteur fixe, on jouer avec **grid-template-rows**

```
.conteneur {  
  display: grid;  
  grid-template-columns: 200px 200px 200px;  
  grid-template-rows: 100px 150px 200px;  
}
```

Gestion des menus avec **grid**

gap

👉 C'est une propriété qui crée de l'espacement entre les bloc

```
.box {  
  height: 150px;  
}  
.conteneur {  
  display: grid;  
  grid-template-columns: 200px 200px 200px;  
  grid-template-rows: 100px 150px 200px;  
  gap : 10px;  
}
```

Gestion des menus avec **grid**

👉 Les unités des blocs :

👉 px ; em ou rem ; % ; **fr**

```
.box {  
  height: 150px;  
}  
.conteneur {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  grid-template-rows: 50px 100px 200px;  
  gap: 10px;  
}
```

Les tableaux : **table**

- 👉 Le tableau est un ensemble de lignes et de colonnes
- 👉 On le met en place avec la balise `<table></table>` qui est une balise de **type block**
- 👉 les lignes sont gérées par la balise `<tr></tr>`
- 👉 les colonnes sont gérées par la balise `<td></td>`

```
<table>
  <tr>
    <td></td>
    <td></td>
  </tr>
  <tr>
    <td></td>
    <td></td>
  </tr>
</table>
```

Les tableaux : **table**

les bordures des cellules : **border**

```
td /* Toutes les cellules des tableaux... */  
{  
  border: 1px solid black; /* auront une bordure de 1px */  
}
```

cellule 1.1	cellule 1.1
cellule 2.1	cellule 2.2

👉 **border-collapse** : un paramètre de table qui permet de coller ou de séparer les cellules

👉 **border-collapse : collapse** ; les cellules collées

👉 **border-collapse : separate** ; les cellules séparées

Structuration des tableaux

- ☞ Une table peut être divisée en trois parties : `thead`, `tbody` et `tfoot`
 - ☞ l'en-tête du tableau (en haut) se définit avec les balises `<thead></thead>`
 - ☞ le corps du tableau (au centre) se définit avec les balises `<tbody></tbody>`
 - ☞ le pied du tableau (en bas) se définit avec les balises `<tfoot></tfoot>`

Les tableaux : **table**

Fusions horizontale : **colspan** et verticale : **rowspan**

- 👉 **colspan** permet de fusionner des colonnes : la fusion s'effectue horizontalement
- 👉 **rowspan** permet de fusionner des lignes : La fusion s'effectuera verticalement

Titre du film	Pour enfants ?	Pour adolescents ?
Massacre à la tronçonneuse	Non, trop violent	Oui
The Ring	Effrayant	
Les bisounours font du ski	Oui, adapté	Pas assez violent...
Lucky Luke, seul contre tous	Pour toute la famille !	

Les formulaires : form

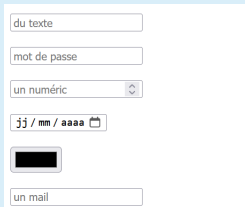
- ☞ Trois attributs sont indispensables pour la balise `<form>`
`</form>`
 - ☞ `method` : la façon où des données seront envoyées
 - ✈ `get`
 - ✈ `post`
 - ☞ `action` : adresse de la page ou du programme qui va traiter les informations. Cette partie sera développée dans un module qui traite la partie `backend`

👉 Dans le formulaire, les informations à traiter seront dans objets :

- 👁 `<input>` : une entrée qui a un attribut `type` qui renseigne sa nature
 - ✈ `type="text"` : pour saisir une chaîne de caractères
 - ✈ `type="password"` : pour saisir un mot de passe
 - ✈ `type="mail"` : pour saisir l'email
 - ✈ `type="numeric"` : une valeur numerique
 - ✈ `type="date"` : une valeur une date
 - ✈ `type="color"` : une couleur
 - ✈ etc.

Les formulaires : form

```
<form action="#" method="post">  
  <input type="text" size="20" placeholder="du texte"> <br><br>  
  <input type="password" size="20" placeholder="mot de passe"> <br><br>  
  <input type="number" size="20" placeholder="un numéric"> <br><br>  
  <input type="date" size="20" placeholder="une date"> <br><br>  
  <input type="color" size="20" placeholder="une couleur"> <br><br>  
  <input type="mail" size="20" placeholder="un mail"> <br>  
</form>
```



du texte

mot de passe

un numéric

jj / mm / aaaa

une couleur

un mail